



PDF

CKJ - SISTEMA DE ALMACÉN

Seguridad, Resiliencia y Contingencia DevOps

Implementación de DevSecOps, alta disponibilidad y planes de recuperación en un sistema de gestión de inventarios con Angular 22, Node.js + Express y SQLite. Despliegue automatizado con Docker, Ansible y PM2.

Angular 22

Node.js + Express

SQLite

Docker

Ansible

DevSecOps

Husky + Gitleaks

← Desliza o usa las flechas del teclado →



Índice

Tres pilares DevOps en el proyecto CKJ



Seguridad / DevSecOps

Implementado: JWT, Helmet, bcrypt, Rate Limit, validación, Husky pre-commit, lint-staged, Gitleaks, npm audit, Trivy, SonarQube, Snyk, Checkov, Fail2Ban, UFW, Hardening SSH

Propuesta: Terraform + AWS WAF, K8s Secrets + Vault, RDS PostgreSQL



Resiliencia

Implementado: PM2 auto-reinicio + watch + 300MB RAM, Docker multi-stage + restart always, Ansible deploy + rollback, Nginx reverse proxy + SSL + caché 1 año

Propuesta: Kubernetes + HPA (3 réplicas), AWS Auto Scaling, ALB



Contingencia

Implementado: saveDb() persistencia inmediata, Rate limiting anti-DoS, UFW firewall (solo 22,80,443), Rollback Ansible (<2min), Hardening preventivo

Propuesta: RDS Multi-AZ + backups 30 días, K8s PodDisruptionBudget, S3 para uploads

PILAR 1

Seguridad & DevSecOps

Seguridad integrada en cada etapa: desde el editor de código (Husky, Gitleaks), pasando por la compilación (npm audit, SonarQube), hasta la infraestructura (Fail2Ban, UFW, Hardening SSH).

Seguridad en el Código

Código real del proyecto CKJ - archivo `backend/server.js`

Helmet + Rate Limiting ☒ CÓDIGO REAL

```
const helmet = require('helmet');
const ratelimit = require('express-rate-limit');
app.use(helmet()); // XSS, clickjacking, MIME sniffing
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, max: 100 // por IP
});
app.use('/api/', limiter);
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, max: 5 // 5 intentos
});
app.post('/api/login', loginLimiter, ...);
```

¿Qué hace? Helmet añade cabeceras HTTP de seguridad (X-Content-Type-Options, X-Frame-Options). El rate limiter evita ataques de fuerza bruta limitando a 100 peticiones/IP cada 15 min, y solo 5 intentos de login.

JWT + Middleware de Roles ☒ CÓDIGO REAL

```
function authMiddleware(rolesPermitidos = []) {
  return (req, res, next) => {
    const token = req.headers['authorization']?.split(' ')[1];
    if (!token) return res.status(401).json({error: 'Token'});
    const decoded = jwt.verify(token, JWT_SECRET);
    const user = queryOne('SELECT * FROM usuarios WHERE id=?', [decoded.id]);
    if (!user) return res.status(401).json({error: 'No válido'});
    if (rolesPermitidos.length && !rolesPermitidos.includes(user.rol))
      return res.status(403).json({error: 'Permisos'});
    req.user = user; next();
  };
}

// Uso por ruta:
app.get('/api/usuarios', authMiddleware(['admin']), ...);
app.post('/materiales', authMiddleware(['admin', 'almacen']), ...);
```

¿Qué hace? Cada ruta declara qué roles pueden acceder. El middleware verifica el token JWT (expira cada 12h), busca el usuario en BD y comprueba su rol. Si no tiene permisos → 403 Forbidden.



bcrypt - Hash de Contraseñas ☒ CÓDIGO REAL

```
const bcrypt = require('bcryptjs');
// Al crear usuario: hash con 10 rondas de sal
const hashed = bcrypt.hashSync(password, 10);
db.run('INSERT INTO usuarios VALUES (NULL,?,?,?,1)',
  [username, hashed, nombre, rol]);
// Al iniciar sesión: comparar hash
bcrypt.compareSync(password, user.password);
```

Nunca almacena contraseñas en texto plano. bcrypt con 10 rondas hace que cada verificación tome ~100ms. Si la BD se ve comprometida, las contraseñas no pueden recuperarse.



Validación + Límite 1MB ☒ CÓDIGO REAL

```
// Límite de tamaño del body: 1MB máximo
app.use(express.json({ limit: '1mb' }));
// Validación estricta de tipos
if (typeof username !== 'string' || typeof password !== 'string')
  return res.status(400).json({error:'Datos inválidos'});
// Validación de rol contra lista blanca
if (!['admin','almacen','ventas'].includes(rol))
  return res.status(400).json({error:'Rol inválido'});
```

Tres capas: límite de 1MB (anti-DoS), validación de tipos (typeof), lista blanca de roles permitidos. Todo esto evita inyección de datos maliciosos.



DevSecOps - Seguridad en el Pipeline

Husky, Gitleaks, escaneos automáticos y hardening - código real del proyecto



Husky + lint-staged + Gitleaks ☒ CÓDIGO REAL

`.husky/pre-commit` - Se ejecuta antes de cada **git commit**

```
# .husky/pre-commit
npx lint-staged
npx --yes gitleaks detect --source . --no-git --verbose

# package.json - lint-staged
"lint-staged": {
  "*.ts,js,json,md,css,html": "prettier --write",
  "*.ts,js": "eslint --fix --quiet"
}
```

¿Qué hace? Husky ejecuta hooks antes de cada commit: 1) lint-staged formatea código con Prettier y corrige errores con ESLint automáticamente. 2) Gitleaks escanea el código en busca de 150+ patrones de secretos (tokens AWS, GitHub, contraseñas, claves SSH). Si encuentra algo sospechoso, bloquea el commit. Esto evita que código mal formateado o con credenciales llegue al repositorio.

Escaneos Automáticos ☒ ANSIBLE REAL

`ansible/devsecops/scan.yml` - 5 herramientas de seguridad

```
# 1. npm audit - Vulnerabilidades en dependencias
- command: npm audit --audit-level=high
  register: npm_audit_backend
# 2. Trivy - Escaneo de archivos (HIGH/CRITICAL)
- command: trivy fs --severity HIGH,CRITICAL
# 3. Checkov - Escaneo de playbooks Ansible
- command: checkov --directory ./ansible
# 4. SonarQube - Calidad y seguridad de código
- command: sonar-scanner
# 5. Snyk - Monitoreo continuo de vulnerabilidades
- command: snyk test --all-projects
```

Herramientas: npm audit (vulnerabilidades npm), Trivy (archivos y contenedores), Checkov (configuración Ansible), SonarQube (calidad de código), Snyk (monitoreo continuo). Todos los resultados se guardan en `/var/log/ckj/scans/`.

Hardening del Servidor ☒ ANSIBLE REAL

`ansible/roles/security/tasks/main.yml` - Configuración completa

```
# Fail2Ban: bloquea IPs tras 3 intentos SSH
- apt: name=fail2ban state=present
# UFW Firewall: denegar todo, permitir solo lo necesario
- ufw: rule=allow port=22 proto=tcp # SSH
- ufw: rule=allow port=80 proto=tcp # HTTP
- ufw: rule=allow port=443 proto=tcp # HTTPS
- ufw: rule=deny port=3000 proto=tcp # API oculta
- ufw: state=enabled policy=deny
# Hardening SSH
- lineinfile: line='PermitRootLogin no'
- lineinfile: line='PasswordAuthentication no'
- lineinfile: line='MaxAuthTries 3'
# Actualizaciones automáticas de seguridad
- apt: name=unattended-upgrades state=present
```

¿Qué hace? Fail2Ban monitorea logs SSH y bloquea IPs tras 3 intentos. UFW solo permite puertos 22 (SSH), 80 (HTTP) y 443 (HTTPS). SSH sin root, sin contraseñas, 3 intentos máx. Actualizaciones de seguridad automáticas.

Protección de archivos sensibles ☒ ANSIBLE REAL

```
# Proteger archivo .env del backend
- file: path={{ backend_dir }}/env
  owner={{ app_user }} mode=0600
# Proteger base de datos SQLite
- file: path={{ db_path }}
  owner={{ app_user }} mode=0600
# Limitar permisos del directorio uploads
- file: path={{ uploads_dir }}
  owner={{ app_user }} mode=0755
# Deshabilitar server_tokens de nginx
- lineinfile: path=/etc/nginx/nginx.conf
  regexp='server_tokens' state=absent
```

Los archivos sensibles (.env, database.sqlite) tienen permisos 0600 (solo lectura/escritura para el dueño). El directorio uploads tiene permisos restringidos. Nginx no muestra su versión (server_tokens off), evitando que atacantes sepan qué versión está corriendo.

Seguridad - Propuesta de Mejora

Herramientas que NO están implementadas en CKJ pero mejorarían la seguridad

Terraform + AWS WAF PROPUESTA

Infraestructura como código con Web Application Firewall en AWS

```
# terraform/waf.tf - ESTO NO ESTÁ EN EL PROYECTO ACTUAL
resource "aws_wafv2_web_acl" "ckj_waf" {
  name = "ckj-waf"; scope = "REGIONAL"
  rule { # Rate limiting a nivel de red
    name = "rate-limit"; priority = 1
    action { block {} }
    statement { rate_based_statement {
      limit = 2000; aggregate_key_type = "IP"
    }}
  }
  rule { # Bloqueo de XSS (Cross-Site Scripting)
    name = "xss-block"; priority = 2
    action { block {} }
    statement { xss_match_statement {
      field_to_match { query_string {} }
    }}
  }
}
```

 **Cambios necesarios en CKJ:** 1) Crear archivos .tf con VPC, subnets, security groups. 2) Migrar BD de SQLite a RDS PostgreSQL (cambiar sql.js por pg). 3) Subir imagen Docker a ECR. 4) Configurar WAF + CloudFront como CDN. 5) Mover uploads/ a S3. **Tiempo estimado:** 2-3 semanas.



Kubernetes Secrets + Vault PROPUESTA

Gestión centralizada y rotación automática de secretos

```
# k8s/secrets.yaml - ESTO NO ESTÁ EN EL PROYECTO
apiVersion: v1
kind: Secret
metadata:
  name: ckj-secrets
type: Opaque
stringData:
  JWT_SECRET: "${JWT_SECRET}" # Desde Vault
  DB_PASSWORD: "${DB_PASSWORD}"

# backend/server.js - YA SOPORTA variables de entorno
const JWT_SECRET = process.env.JWT_SECRET
|| 'ckj-almacen-secret-2026'; // fallback local
```

 **Cambios necesarios en CKJ:** 1) El código backend ya lee JWT_SECRET de variable de entorno (process.env). Solo falta crear los objetos Secret en Kubernetes y conectarlos al Deployment. 2) Implementar HashiCorp Vault Agent para rotación automática de secretos cada 30 días. **Tiempo estimado:** 1-2 semanas.

PILAR 2



Resiliencia del Sistema

La resiliencia es la capacidad del sistema de mantener su funcionamiento ante fallos, picos de tráfico o condiciones adversas. Múltiples capas de auto-recuperación garantizan que el servicio esté disponible incluso cuando algo sale mal.



Resiliencia Implementada en CKJ

PM2 (auto-reinicio), Docker (multi-stage + restart), Ansible (rollback), Nginx (caché)



PM2 - Process Manager ☒ ANSIBLE REAL

ansible/roles/app/tasks/main.yml

```
# PM2: auto-reinicio + límite de memoria
pm2 start {{ backend_dir }}/server.js \
  --name ckj-api \
  --watch # Reinicia si cambia código
  --ignore-watch="*.sqlite" # No reinicia por BD
  --max-memory-restart 300M # Reinicia si excede 300MB

# Inicio automático al encender servidor
```

```
pm2 startup systemd          # Crea servicio systemd
pm2 save                     # Persiste configuración
```

 Fallo →  PM2 detecta →  Reinicia en <2s

¿Cómo funciona? Si el proceso se cae por error o memory leak, PM2 reinicia en menos de 2 segundos. El modo watch detecta cambios en código y reinicia la app automáticamente. PM2 startup systemd arranca la app al iniciar el servidor. El límite de 300MB evita que un leak consuma toda la RAM.

Docker Multi-Stage ☒ DOCKERFILE REAL

Dockerfile - Imagen mínima de ~300MB (vs ~1.5GB)

```
FROM node:22-alpine AS build-frontend
WORKDIR /app
COPY package*.json ./; RUN npm ci --only=production
COPY . .; RUN npm run build -- --configuration production
```

```
FROM node:22-alpine
# Contenido final: solo lo necesario
RUN apk add --no-cache tini      # Manejo SIGTERM
RUN addgroup -S ckj && adduser -S ckj -G ckj
USER ckj                        # Seguridad (no-root)
EXPOSE 3000
ENTRYPOINT ["/sbin/tini", "--"]
CMD ["node", "backend/server.js"]

# docker run -d -p 3000:3000 --restart always ckj:latest
```

Beneficio: La imagen final es ~300MB vs ~1.5GB. --restart always reinicia automáticamente si el contenedor falla. Tini maneja correctamente señales SIGTERM para shutdown graceful. Usuario no-root (ckj) limita permisos del contenedor.

Ansible Rollback ☒ ANSIBLE REAL

ansible/rollback.yml - Recuperación en <2 minutos

```
# Volver a versión anterior
ansible-playbook rollback.yml -e "git_ref=v1.0.0"

# El playbook ejecuta automáticamente:
# 1. git checkout a versión estable
# 2. Reinstala dependencias (npm ci)
# 3. Recompila Angular (ng build --prod)
# 4. pm2 restart ckj-api
# Tiempo total de recuperación: < 2 minutos
```

 Bug en prod ↓  Ejecutar rollback.yml ↓ ☒ Versión estable restaurada



Nginx Reverse Proxy ☒ ANSIBLE REAL

ansible/roles/nginx/templates/ckj.nginx.conf.j2

```
server {
    listen 443 ssl http2;
    server_name {{ nginx_server_name }};
    # SSL con TLSv1.2 y TLSv1.3
    ssl_certificate {{ ssl_cert_path }};
    ssl_certificate_key {{ ssl_key_path }};
    # Frontend Angular - caché 1 año
    location / {
        root {{ frontend_dir }};
        try_files $uri $uri/ /index.html;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
    # API Backend (proxy inverso)
    location /api/ {
        proxy_pass http://127.0.0.1:{{ backend_port }};
    }
    # Seguridad: bloquear rutas sensibles
    location ~ /\. { deny all; }
    location ~ /(node_modules|.git|src) { deny all; }
}
```

¿Qué hace? Nginx actúa como reverse proxy: el frontend se cachea por 1 año (los usuarios siguen viendo la página aunque el backend tenga problemas). SSL/HTTPS cifra toda la comunicación. HTTP/2 mejora rendimiento. Bloquea acceso a rutas sensibles (.git, node_modules, src).



Resiliencia - Propuesta de Mejora

Alta disponibilidad con Kubernetes, HPA y AWS Auto Scaling



Kubernetes Deployment + HPA ☒ PROPUESTA

Orquestación de contenedores con auto-recuperación y escalado horizontal

```
# k8s/deployment.yaml - ESTO NO ESTÁ EN EL PROYECTO
apiVersion: apps/v1
kind: Deployment
metadata: { name: ckj-api }
spec:
  replicas: 3 # Múltiples instancias para alta disponibilidad
  selector: { matchLabels: { app: ckj } }
  template:
    metadata: { labels: { app: ckj } }
    spec:
      containers:
```



```

- name: ckj-api
  image: ckj:latest
  ports: [{ containerPort: 3000 }]
  resources:
    limits: { memory: "256Mi", cpu: "500m" }
  livenessProbe: # 💖 ¿El proceso está vivo?
    httpGet: { path: /materiales, port: 3000 }
    initialDelaySeconds: 15
  readinessProbe: # ✅ ¿Está listo para recibir tráfico?
    httpGet: { path: /dashboard, port: 3000 }
    initialDelaySeconds: 5

# HPA: Escalado automático horizontal
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: ckj-api-hpa }
spec:
  minReplicas: 3; maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization, averageUtilization: 60 }

```

🔑 **Cambios en CKJ:** 1) Ya dockerizado (Dockerfile listo). 2) Crear manifiestos YAML. 3) Migrar BD SQLite a RDS PostgreSQL. 4) Configurar Ingress Controller. 5) Uploads a S3. **Beneficio:** Si un pod falla, K8s lo recrea. Si CPU > 60%, HPA crea más pods. **Tiempo:** 3-4 semanas.

📊 AWS Auto Scaling + ALB ➡ PROPUESTA

Escalado automático de instancias EC2 según la demanda en AWS

```

# terraform/autoscaling.tf - ESTO NO ESTÁ EN EL PROYECTO
resource "aws_launch_template" "ckj" {
  image_id = data.aws_ami.amazon_linux_2.id
  instance_type = "t3.medium"
  user_data = base64encode("#!/bin/bash
    docker run -d -p 3000:3000 --restart always ckj:latest")
}
resource "aws_autoscaling_group" "ckj_asg" {
  min_size = 2; max_size = 10; desired_capacity = 2
  launch_template { id = aws_launch_template.ckj.id }
  target_group_arns = [aws_lb_target_group.ckj.arn]
}
resource "aws_autoscaling_policy" "ckj_cpu" {
  policy_type = "TargetTrackingScaling"
  target_tracking_configuration {
    predefined_metric_specification {
      predefined_metric_type = "ASGAverageCPUUtilization"
    }
  }
}

```

```
target_value = 60.0 # Escala si CPU > 60%
}
}
```

🔧 **Cambios en CKJ:** 1) Migrar a RDS PostgreSQL. 2) Crear AMI personalizada con Docker y la app. 3) Configurar ALB para distribuir tráfico. 4) Uploads a S3. **Escenario:** Black Friday: CPU > 60% → Auto Scaling lanza instancias (2→10). Cuando baja la demanda, las elimina. **Tiempo:** 4-6 semanas.

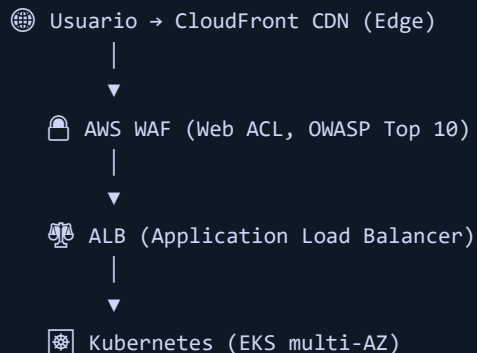
Diagrama de Arquitectura

Estado actual del proyecto vs Propuesta con Kubernetes + AWS

ARQUITECTURA ACTUAL



ARQUITECTURA PROPUESTA



```
| Pods ckj-api: 3 por zona |  
| (us-east-1a, 1b, 1c)   |
```



 RDS PostgreSQL Multi-AZ
(backup automático 30 días)

 Fallos adicionales:

- ✗ Pod crashea → K8s lo recrea ☒
- ✗ CPU >60% → HPA escala a +pods ☒
- ✗ Zona completa cae → Multi-AZ ☒
- ✗ BD corrupta → RDS restore ☒
- ✗ DDoS → WAF + CloudFront ☒

PILAR 3

Contingencia & Disaster Recovery

La contingencia son los planes para recuperar el sistema ante un desastre: pérdida de datos, caída del servidor, ataque cibernético o error humano. Un buen plan minimiza el RTO (tiempo de recuperación) y el RPO (pérdida de datos).

Contingencia Implementada en CKJ

saveDb() persistencia inmediata, Rate limiting, UFW, Rollback, Hardening

Persistencia Inmediata (Backup continuo) ☒ CÓDIGO REAL

`backend/server.js` - Cada operación SQL se guarda al instante

```
function saveDb() {  
  // Exporta TODA la BD a disco tras cada cambio  
  const data = db.export();  
  fs.writeFileSync(DB_PATH, Buffer.from(data));  
}  
function runSql(sql, params = []) {  
  db.run(sql, params);  
  saveDb(); // ¡Persistencia en CADA operación!  
}
```

¿Qué hace? Cada INSERT, UPDATE o DELETE ejecuta saveDb(), que exporta toda la base de datos al archivo database.sqlite. Si el servidor se apaga inesperadamente, solo se pierde la operación en

curso. Es un backup continuo en tiempo real. El archivo puede respaldarse externamente con un simple copy.

Rate Limiting (Anti-DoS) CÓDIGO REAL

`backend/server.js` - Protección contra denegación de servicio

```
// Límite general: 100 peticiones cada 15 min por IP
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, max: 100,
  standardHeaders: true, legacyHeaders: false,
  message: { error: 'Demasiadas solicitudes' }
});
app.use('/api/', limiter);
// Límite estricto login: solo 5 intentos cada 15 min
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, max: 5,
  message: { error: 'Demasiados intentos' }
});
app.post('/api/login', loginLimiter, ...);
```

¿Qué hace? Si un atacante intenta saturar la API, responde con error 429 Too Many Requests y bloquea esa IP temporalmente. El login solo permite 5 intentos cada 15 minutos, bloqueando ataques de fuerza bruta. Las cabeceras estándar informan al cliente sobre su límite actual.

Hardening Preventivo ANSIBLE REAL

`ansible/roles/security/tasks/main.yml`

```
# UFW: Denegar TODO por defecto
- ufw: state=enabled direction=incoming policy=deny
# Solo permitir lo esencial:
- { rule: allow, port: '22', proto: tcp } # SSH
- { rule: allow, port: '80', proto: tcp } # HTTP
- { rule: allow, port: '443', proto: tcp } # HTTPS
- { rule: deny, port: '3000', proto: tcp } # API oculta
# SSH Hardening
PermitRootLogin no
PasswordAuthentication no
MaxAuthTries 3
# Fail2Ban - Bloqueo automático
[sshd]
enabled = true; maxretry = 3; bantime = 3600
```

¿Qué hace? Principio de "denegar todo por defecto". UFW bloquea todo excepto SSH (22), HTTP (80) y HTTPS (443). La API en :3000 no está expuesta directamente (solo via Nginx). SSH sin root, sin contraseñas, 3 intentos máx. Fail2Ban bloquea IPs maliciosas por 1 hora.

Rollback Ansible ANSIBLE REAL

ansible/rollback.yml - Plan de contingencia

```
# Plan: ¿Qué hago si la nueva versión tiene un bug?
# Paso 1: Ejecutar rollback
ansible-playbook rollback.yml -e "git_ref=v1.0.0"

# El playbook ejecuta automáticamente:
- git checkout {{ git_ref }}      # Volver a código anterior
- npm ci                          # Reinstalar dependencias
- npm run build -- --prod         # Recompilar frontend
- pm2 restart ckj-api            # Reiniciar backend
# Tiempo total de recuperación: < 2 minutos
```

¿Qué hace? Permite revertir a una versión anterior del código en menos de 2 minutos. Es el plan de recuperación principal ante un bug crítico. El flujo es automático: checkout, dependencias, compilación y reinicio. El playbook status.yml verifica que la recuperación fue exitosa.



Contingencia - Propuesta de Mejora

Disaster Recovery profesional en la nube con AWS



RDS Multi-AZ + Backup Automático PROPUESTA

Base de datos redundante en múltiples zonas de disponibilidad

```
# terraform/rds.tf - ESTO NO ESTÁ EN EL PROYECTO ACTUAL
resource "aws_db_instance" "ckj_db" {
  engine           = "postgres"
  instance_class   = "db.t3.medium"
  multi_az         = true           # Réplica en otra zona
  backup_retention_period = 30      # Backups 30 días
  backup_window    = "03:00-04:00"
  deletion_protection = true       # No eliminar accidental
  copy_tags_to_snapshot = true
}

// 🛠 Cambios necesarios en CKJ:
// 1. Reemplazar require('sql.js') por require('pg')
// 2. Las consultas SQL actuales son compatibles
// 3. Agregar pool de conexiones (pg-pool)
// 4. Configurar vía variables de entorno
```

Beneficio: Si la BD primaria falla, RDS hace failover a la réplica Multi-AZ en <60 segundos sin pérdida de datos. Backups automáticos permiten restaurar a cualquier punto en los últimos 30 días. **Tiempo:** 2-3 semanas.



K8s Disaster Recovery Multi-Zona PROPUESTA

Plan de recuperación ante desastres con Kubernetes multi-zona

```
# k8s/pod-disruption-budget.yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: ckj-pdb }
spec:
  minAvailable: 2           # Mínimo 2 pods siempre activos
  selector: { matchLabels: { app: ckj } }

# Distribuir pods entre zonas de disponibilidad
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule

// 🛠 Cambios necesarios en CKJ:
// 1. EKS multi-zona (us-east-1a, 1b, 1c)
// 2. BD: RDS Multi-AZ (no SQLite Local)
// 3. Uploads: S3 en vez de disco local
// 4. Frontend: CloudFront + S3 bucket
```

Beneficio: PodDisruptionBudget garantiza que siempre haya al menos 2 pods funcionando. TopologySpreadConstraints asegura que los pods se distribuyan en distintas zonas de disponibilidad. Si una zona completa falla, las otras siguen sirviendo. **Tiempo:** 4-5 semanas.



Tabla Comparativa Completa

20 herramientas implementadas en CKJ + 8 propuestas de mejora para escalar a la nube

#	Herramienta	Propósito	¿Dónde?	Estado
1	Helmet	Cabeceras HTTP seguras	server.js	✅ CÓDIGO
2	Rate Limiting	100 req/15min, 5 login	server.js	✅ CÓDIGO
3	JWT + RBAC	Tokens 12h + roles	server.js	✅ CÓDIGO
4	bcrypt	Hash 10 rondas sal	server.js	✅ CÓDIGO
5	Validación	Tipos + 1MB + lista blanca	server.js	✅ CÓDIGO
6	Husky	Pre-commit hooks	.husky/	✅ CÓDIGO
7	lint-staged	Prettier + ESLint auto	package.json	✅ CÓDIGO

8	Gitleaks	Detección secretos (150+ patrones)	.husky/pre-commit	✓ CÓDIGO
9	npm audit	Vulnerabilidades npm	ansible/scan.yml	✓ ANSIBLE
10	Trivy	Escaneo archivos	ansible/scan.yml	✓ ANSIBLE
11	SonarQube	Calidad código	ansible/scan.yml	✓ ANSIBLE
12	Snyk	Monitoreo vulnerabilidades	ansible/scan.yml	✓ ANSIBLE
13	Checkov	Escaneo Ansible	ansible/scan.yml	✓ ANSIBLE
14	PM2	Auto-reinicio + 300MB	ansible/app/	✓ ANSIBLE
15	Docker	Multi-stage build	Dockerfile	✓ DOCKER
16	Ansible	Deploy + rollback	ansible/	✓ ANSIBLE
17	Nginx	SSL + caché + proxy	ansible/nginx/	✓ ANSIBLE
18	Fail2Ban	Bloqueo SSH	ansible/security/	✓ ANSIBLE
19	UFW	Firewall 22,80,443	ansible/security/	✓ ANSIBLE
20	saveDb()	Persistencia inmediata	server.js	✓ CÓDIGO
21	Terraform	IaC en AWS	—	🔗 PROP.
22	AWS WAF	Firewall web OWASP	—	🔗 PROP.
23	Kubernetes	Orquestación + HPA	—	🔗 PROP.
24	AWS Auto Scaling	Escalado CPU > 60%	—	🔗 PROP.
25	RDS Multi-AZ	BD redundante	—	🔗 PROP.
26	S3	Uploads escalables	—	🔗 PROP.
27	CloudFront	CDN frontend	—	🔗 PROP.
28	HashiCorp Vault	Rotación secretos	—	🔗 PROP.

Conclusión

Estado actual del proyecto CKJ y hoja de ruta hacia la nube

✓ **Logros actuales (20 medidas)**

El proyecto CKJ implementa actualmente:

-  **8 medidas de seguridad** en código: JWT, Helmet, bcrypt, Rate Limit, validación, Husky, lint-staged, Gitleaks
-  **5 herramientas DevSecOps** en Ansible: npm audit, Trivy, SonarQube, Snyk, Checkov
-  **4 medidas de resiliencia**: PM2, Docker, Ansible rollback, Nginx
-  **3 medidas de contingencia**: saveDb(), UFW+Fail2Ban, Hardening SSH

Total: **20 herramientas** implementadas en un proyecto real funcionando con 51 productos en inventario.

Hoja de ruta propuesta

Para escalar CKJ a nivel profesional en la nube:

Fase	Tiempo	Qué
1	1-2 sem	Terraform + S3 uploads
2	2-3 sem	RDS PostgreSQL
3	3-4 sem	Kubernetes + HPA
4	4-6 sem	AWS Auto Scaling
5	6-8 sem	Disaster Recovery

La combinación de buenas prácticas, automatización e infraestructura robusta permite construir sistemas realmente resilientes y altamente disponibles.

¡Gracias!

CKJ - Sistema de Almacén

Seguridad, Resiliencia y Contingencia en DevOps



20

implementadas



8

propuestas



13

seguridad



Husky

+ Gitleaks

CKJ - Sistema de Almacén © 2026

